

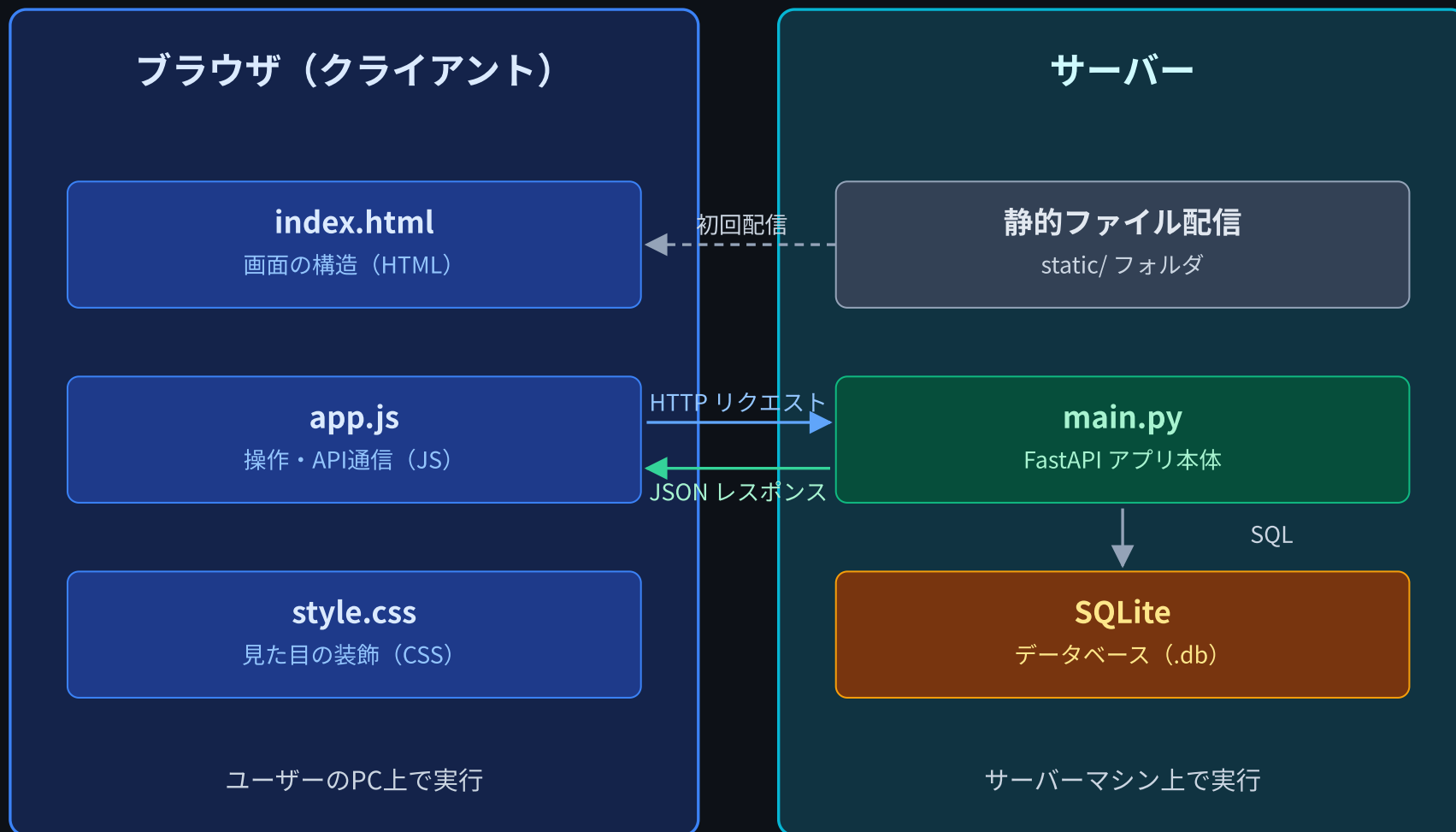
第7回: フロントエンドとバックエンドの結合

Webアプリケーション基礎 2026

今日のゴール

フロントエンドとバックエンドを繋ぎ、フルスタックTODOアプリを完成させる

- 1つのサーバーでHTMLもAPIも配信する（同一オリジン）
- フロントエンドからバックエンドのAPIを呼び出す



今日の流れ

前半

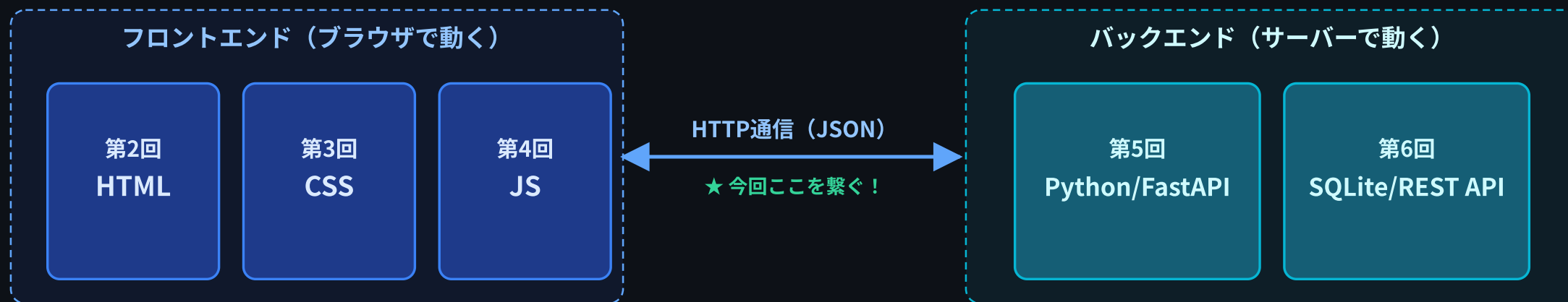
- 全体アーキテクチャ総整理
- 静的ファイル配信 — StaticFiles でHTMLも配信（同一オリジン）
- Fetch API（GET） — `fetch`, `async/await`, `.json()`

後半

- Fetch API（POST） — `method`, `headers`, `body`
- 完了・削除の実装 — `PUT`, `DELETE`
- CORS（発展） — フロントを別オリジンに分けたとき

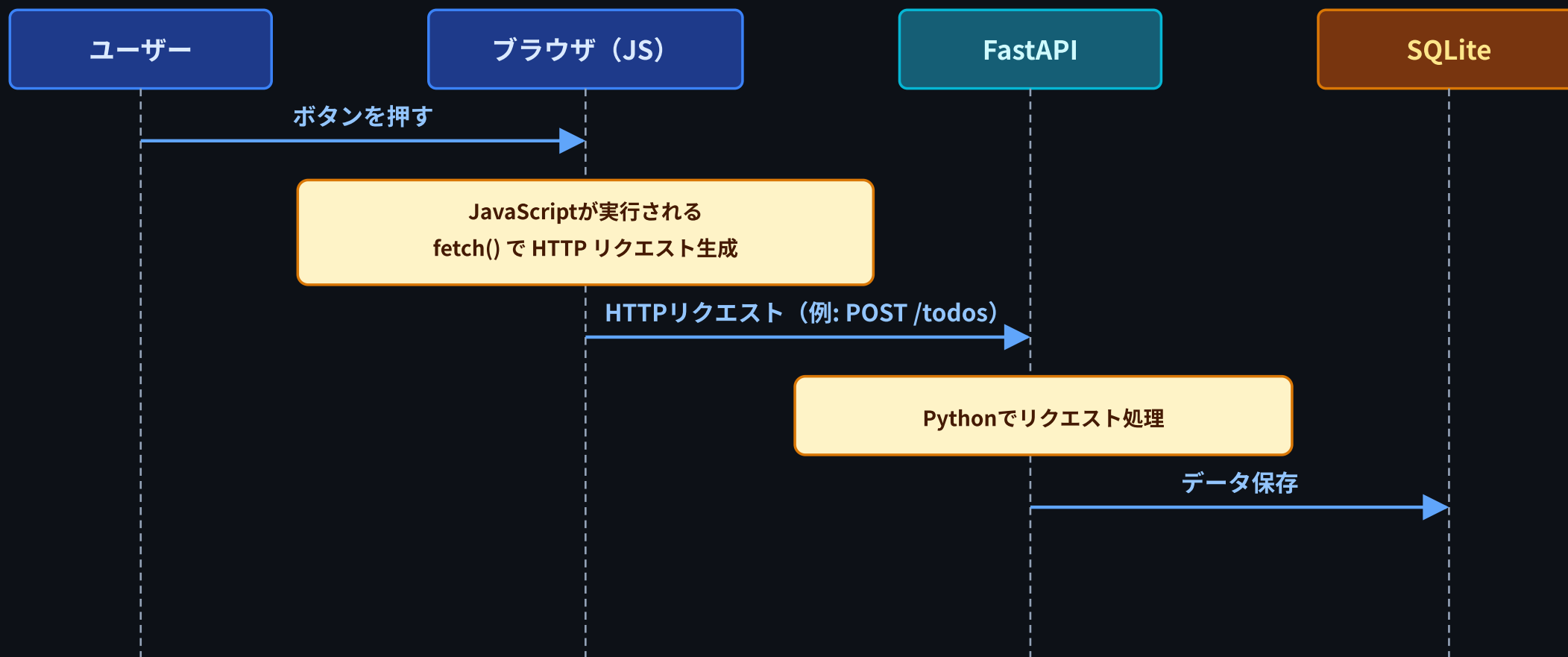
1. 全体アーキテクチャ総整理

これまでの6回で学んだ技術の全体像

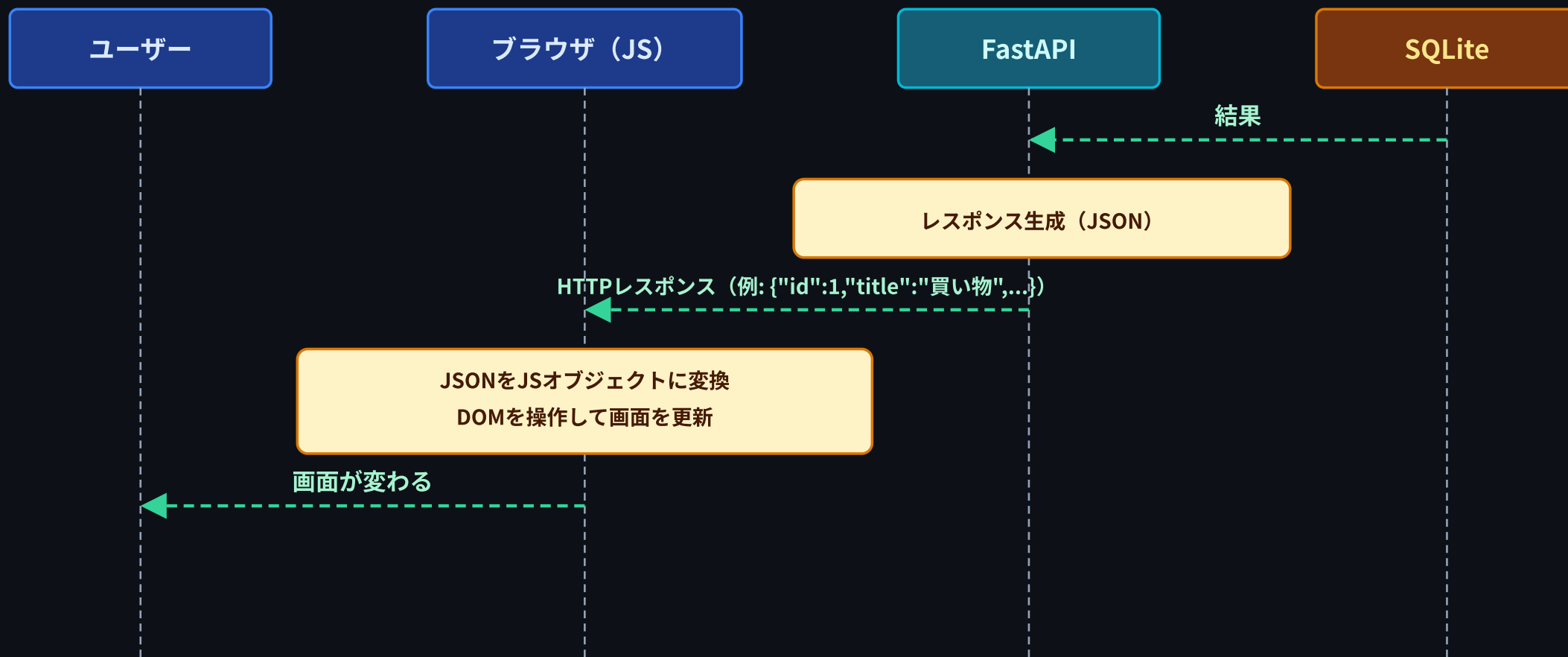


今回は「繋ぐ」回です。

データの流れ -- 全体図（前半）



データの流れ -- 全体図（後半）



各ステップの実行場所

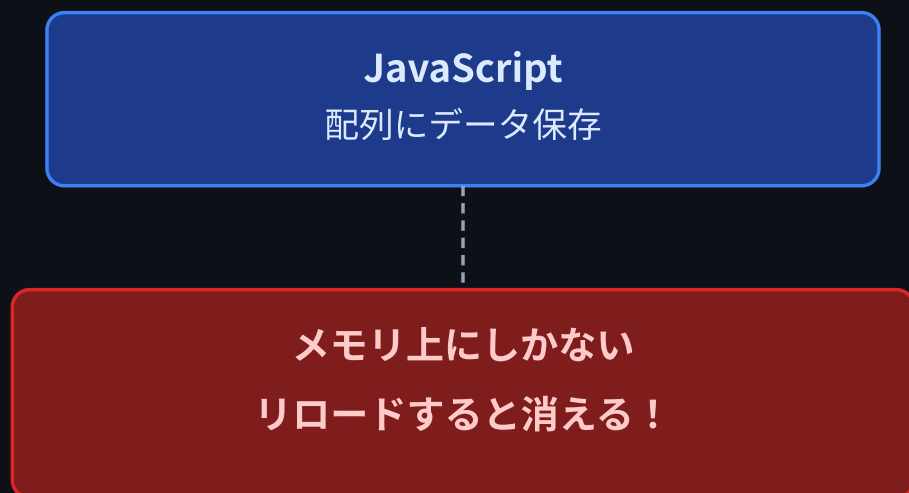
ステップ	何をする	実行場所	使う技術
1	ユーザーがボタンをクリック	ブラウザ	HTML
2	イベントハンドラが発火	ブラウザ	JavaScript
3	HTTPリクエストを送信	ブラウザ → サーバー	Fetch API
4	リクエストを受信・処理	サーバー	FastAPI (Python)
5	データベース操作	サーバー	SQLite (SQL)
6	JSONレスポンスを返す	サーバー → ブラウザ	FastAPI (JSON)
7	レスポンスを受け取る	ブラウザ	Fetch API
8	画面を更新する	ブラウザ	JavaScript (DOM)

ポイント: ステップ3～6の間、ブラウザは「待っている」状態（非同期処理）

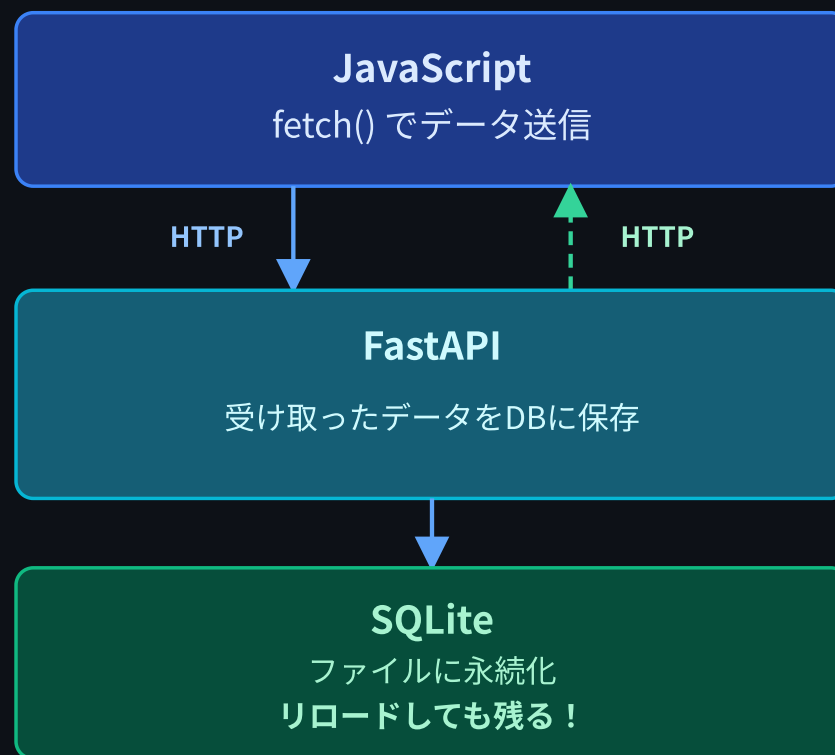
なぜ「繋ぐ」必要があるのか

第4回の問題を思い出そう

フロントエンドだけの場合



バックエンドがある場合



実習1: 通信フローの列挙

やること

以下のシナリオについて、通信フローを列挙してください。

シナリオ: ユーザーが「牛乳を買う」と入力して追加ボタンを押す

1. テキストエディタでファイル `flow.md` を編集
2. 以下の各ステップを書き出す:
 - どこで何が起きるか
 - 実行場所は「ブラウザ」「サーバー」のどれか
 - 使う技術は何か

例: ・サーバー上で新しいTODOをSQLiteでテーブルに保存する

ヒント: 前のスライドの表を参考に、自分の言葉で書いてみましょう

2. 静的ファイル配信

画面（HTML/CSS/JS）はどこから来るのか

これまで:

HTMLファイルを自分でブラウザにドラッグして開いていた
→ 画面（HTML/JS）とAPIサーバーが別々の場所にある

これから:

FastAPIサーバーが HTML/CSS/JS も配信する
→ / で画面（index.html）が表示される
→ /todos で API も使える
→ 画面もAPIも「同じサーバー = 同一オリジン」から届く

静的ファイル配信 = HTML/CSS/JSファイルをFastAPIサーバーから返す仕組み。
まず土台として、サーバーが画面そのものを配信できるようにします。

FastAPIの StaticFiles 設定

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()

# 静的ファイル配信の設定
# "static" フォルダ内のファイルを "/" パスで配信する
# html=True により、/ アクセス時に index.html を自動で返す
app.mount("/", StaticFiles(directory="static", html=True), name="static")
```

注意: `app.mount("/")` はすべてのパスを受け取るため、**APIの定義より後（ファイルの最後）** に書く。

ファイル構成と配信の対応

```
exercise/
├── main.py           ← FastAPIサーバー
├── init_db.py        ← DB初期化
├── todo.db           ← SQLiteデータベース
├── static/           ← このフォルダを配信する
│   ├── index.html    → /                (アプリ画面)
│   ├── style.css     → /style.css
│   └── app.js         → /app.js
```

ファイル	配信されるパス
static/index.html	/
static/style.css	/style.css
static/app.js	/app.js
APIエンドポイント	/todos

画面もAPIも、すべて同じサーバー（同一オリジン）から配信される。

同一オリジンだから fetch は相対パスでOK



```
// 画面とAPIが別サーバーなら、  
// サーバーのURLを全部書く必要がある：  
fetch("http://localhost:8000/todos");  
  
// 同じサーバー（同一オリジン）なら、  
// 相対パスでOK：  
fetch("/todos");  
//      ↑ アドレスを省略できる！
```

画面もAPIも同じサーバーから届くので、**相対パス** `/todos` でAPIを呼べる。
シンプルで、動かす環境（ポート転送など）が変わっても動くため推奨される書き方。
この後の Fetch API は、すべてこの相対パスで書いていきます。

実習2: 静的ファイル配信の設定

やること

準備

1. ターミナルでDBを初期化してサーバーを起動:

```
cd session07/exercise
python init_db.py          # DB初期化
python main.py
```

Step 1: 静的ファイル配信の設定

2. `exercise/main.py` の静的ファイル配信の TODO コメントを実装:

```
app.mount("/", StaticFiles(directory="static", html=True), name="static")
```

(`app.mount` はファイルの最後に書く)

実習2: 静的ファイル配信の設定 (続き)

Step 2: 動作確認

3. サーバーを再起動 (Ctrl+C → `python main.py`)
4. ポートタブを開き、転送された 8000 番のアドレスから「ブラウザで開く」
5. TODOアプリの画面が表示されることを確認
6. この時点で 一覧の取得 (GET) と追加 (POST) は動く
(完了・削除 (PUT/DELETE) は実習5で実装します)

ポイント: 画面 (`index.html / app.js`) も `/todos` API も、同じサーバーから届く = 同一オリジン

3. Fetch API (GET)

配信された画面が、同じサーバーのAPIを呼ぶ

静的配信で 画面がサーバーから届く ようになりました。

次は、その画面の中の JavaScript が 同じサーバーのAPI を呼び出してデータをやり取りします。

Fetch API とは:

- ブラウザに標準搭載されている機能
- JavaScriptからHTTPリクエストを送信できる
- 第1回で学んだHTTP通信を、プログラムから実行する

開発者ツールで手動確認	→	第1回～第6回の方法
Swagger UIでAPIテスト	→	第6回の方法
JavaScriptから自動で通信	→	★今回学ぶ方法 (Fetch API)

非同期処理とは

料理の注文に例えると

同期処理（もし通信が同期だったら）：

注文する → 料理が届くまで何もできない → 料理を受け取る → 食べる
（画面がフリーズしてしまう！）

非同期処理（実際の通信）：

注文する → 待っている間に他のことができる → 料理が届いたら食べる
（画面は操作できたまま、裏で通信が進む）

JavaScriptのHTTP通信は「非同期」 -- ブラウザがフリーズしないように、レスポンスを待つ間も他の処理が実行できる。

fetch() の基本

GETリクエストの書き方

```
// 方法1: .then() チェーン
fetch("/todos")
  .then(response => response.json()) // レスポンスをJSONとして解析
  .then(data => {
    console.log(data);              // データを使う
  });

// 方法2: async/await (こちらを推奨)
async function getTodos() {
  const response = await fetch("/todos");
  const data = await response.json();
  console.log(data);
}
```

await = 「結果が届くまで待つ」という意味

async/await を理解する

```
async function getTodos() {  
    const response = await fetch(..);  
    const data = await response.json();  
    console.log(data);  
}
```

← async: 「この関数は非同期処理を含む」
という宣言

← await: 「サーバーからの返事を待つ」
待っている間、ブラウザは固まらない

← await: 「JSONへの変換を待つ」

← データが届いてから実行される

async と await はセット -- `await` は `async` 関数の中でしか使えない

`response.json()` -- HTTPレスポンスの本文をJSONとして解析し、JavaScriptオブジェクト（配列や辞書）に変換する

実習3: Fetch APIでGETリクエスト

やること

サーバーが起動した状態で（実習2の続き）、

1. ポートタブの 8000 番の転送アドレスを開き、URLの末尾に `/fetch-get-example.html` を足してアクセス
2. 「TODOを取得」ボタンをクリック
3. 開発者ツール → **Console** でデータが表示されることを確認
4. 開発者ツール → **Network** タブでHTTPリクエストを確認
 - 。リクエストURL (`/todos`)、メソッド (GET)、レスポンス (JSON) を観察

ポイント: このデモも `static/` から配信されるので、`fetch("/todos")` という相対パスでAPIに届く

4. Fetch API (POST)

データをサーバーに送信する

GETは「データを取得する」だけでしたが、POSTは「データを送信する」ためのメソッドです。

```
async function addTodo(title) {  
  const response = await fetch("/todos", {  
    method: "POST", // HTTPメソッドを指定  
    headers: {  
      "Content-Type": "application/json" // 送るデータの形式を宣言  
    },  
    body: JSON.stringify({ title: title }) // JavaScriptオブジェクトを  
  }); // JSON文字列に変換して送る  
  const data = await response.json();  
  console.log("追加されたTODO:", data);  
}
```

GET vs POST の違い

GET（データ取得）：

```
fetch("/todos")
```

→ リクエスト本文なし

→ サーバーから「既存のデータ」を受け取る

POST（データ送信）：

```
fetch("/todos", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ title: "買い物" })  
})
```

→ リクエスト本文あり（JSON）

→ サーバーに「新しいデータ」を送る

	GET	POST
目的	データ取得	データ送信
body	なし	あり（JSON）
headers	不要	Content-Type必要

JSON.stringify() とは

JavaScriptオブジェクトとJSON文字列の変換

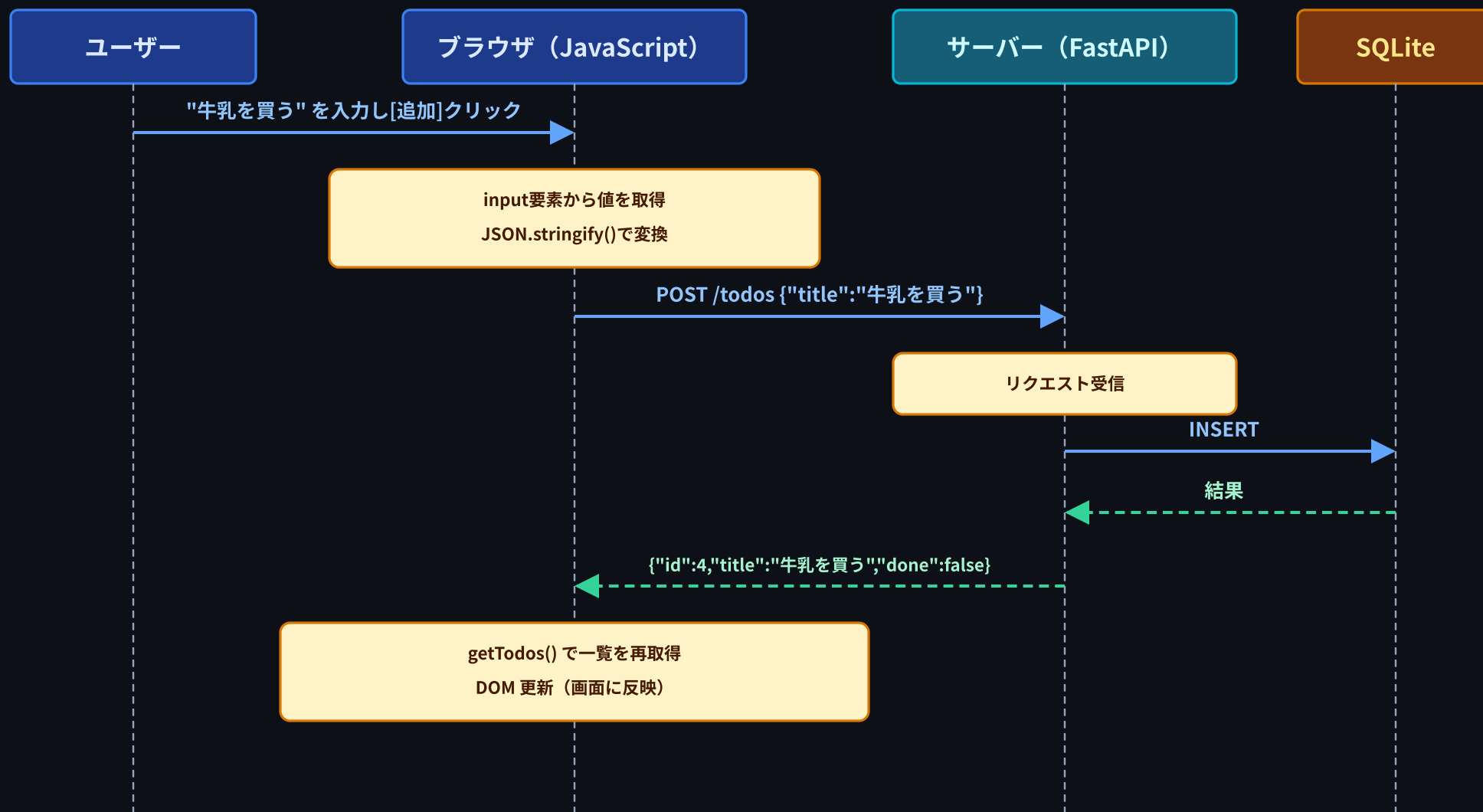
```
// JavaScriptオブジェクト（プログラム内で使う形）
const todo = { title: "牛乳を買う" };

// JSON文字列に変換（サーバーに送る形）
const jsonString = JSON.stringify(todo);
// → '{"title":"牛乳を買う"}'

// 逆変換：JSON文字列 → JavaScriptオブジェクト
const obj = JSON.parse('{"title":"牛乳を買う"}');
// → { title: "牛乳を買う" }
```

JavaScriptオブジェクト	←—JSON.parse()—	JSON文字列
{ title: "..."} ↑プログラム内	—JSON.stringify()→	'{"title":"..."}' ↑通信用

追加ボタンの処理フロー



実習4: Fetch APIでPOSTリクエスト

やること

1. 8000 番の転送アドレスの末尾に `/fetch-post-example.html` を足してアクセス
2. テキスト入力欄に TODO を入力し、「追加」ボタンをクリック
3. **Console** にサーバーからのレスポンスが表示されることを確認
4. **Network** タブで確認:
 - リクエストメソッドが **POST** であること
 - リクエストヘッダに **Content-Type: application/json** があること
 - リクエスト本文 (Payload) に入力した内容がJSON形式で含まれること
 - レスポンスに `id` が付与されて返ってきていること
5. 「TODOを取得」ボタンを押して、追加したTODOが一覧に含まれていることを確認

5. 完了・削除のフロントエンド実装

第4回からの変更点: 配列のindex → DBのid

第4回ではメモリ上の配列を使ったので `todos[index]` で対象を特定していました。
今回からは TODO がDBに保存されるため、ブラウザを閉じてでも消えない**永続的な id** を
DBが自動で割り振ります。エンドポイントも `/todos/{id}` のように id で対象を指定します。

```
// 第4回: 配列のindexで指定
deleteTodo(index); // todos[index] を消す

// 第7回: DBのidで指定
deleteTodo(todo.id); // /todos/{id} に DELETE を送る
```

PUT（完了切替）とDELETE（削除）

```
// PUT: TODO完了状態の切替
async function toggleTodo(id, currentDone) {
  await fetch(`/todos/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ done: !currentDone }) // 現在の反対にする
  });
  await getTodos(); // 一覧を再取得して画面を更新
}

// DELETE: TODOの削除
async function deleteTodo(id) {
  await fetch(`/todos/${id}`, {
    method: "DELETE"
  });
  await getTodos(); // 一覧を再取得して画面を更新
}
```

4つのHTTPメソッドとFetch API

CRUD操作の完全な対応表

操作	HTTPメソッド	エンドポイント	fetch()の書き方
一覧取得	GET	/todos	<code>fetch("/todos")</code>
新規追加	POST	/todos	<code>fetch("/todos", {method:"POST", ...body})</code>
完了切替	PUT	/todos/{id}	<code>fetch("/todos/1", {method:"PUT", ...body})</code>
削除	DELETE	/todos/{id}	<code>fetch("/todos/1", {method:"DELETE"})</code>

パターン:

- **GET/DELETE:** body不要
- **POST/PUT:** headers + body が必要 (JSON形式のデータを送る)

画面更新の流れ

操作のたびに一覧を再取得する（前半）



画面更新の流れ

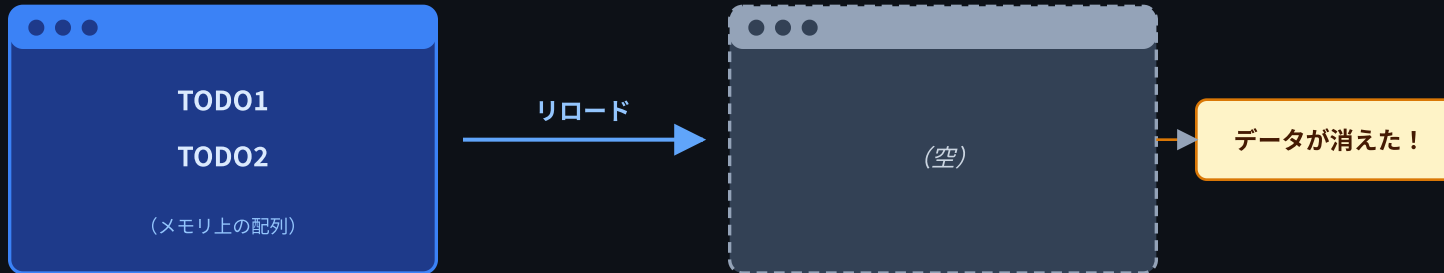
操作のたびに一覧を再取得する（後半）



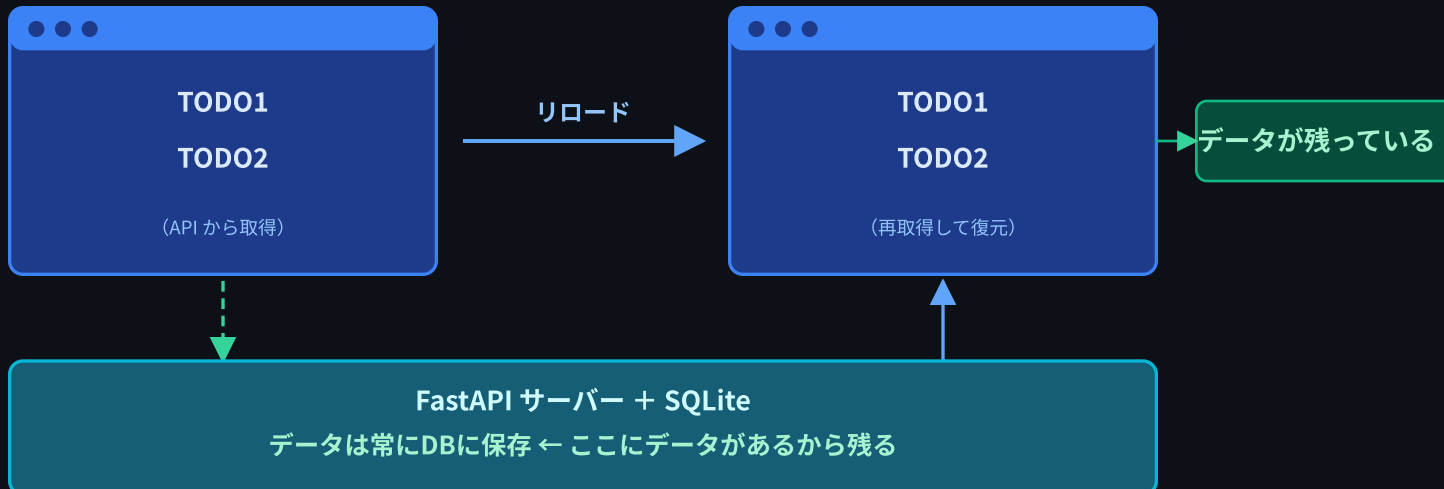
ポイント: 操作が完了したら 必ずサーバーから最新の一覧を取得し直す。
これにより、画面とサーバーのデータが常に同期される。

リロードしてもデータが残る！

これまで（第4回: フロントエンドのみ）



今回（フルスタック版）



実習5: PUT/DELETEの実装

やること

1. `exercise/static/app.js` を開く
2. `toggleTodo` 関数の TODO コメント部分を実装:

```
await fetch(`/todos/${id}`, {  
  method: "PUT",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ done: !done })  
});
```

3. `deleteTodo` 関数の TODO コメント部分を実装:

```
await fetch(`/todos/${id}`, { method: "DELETE" });
```

実習5: PUT/DELETEの実装（続き）

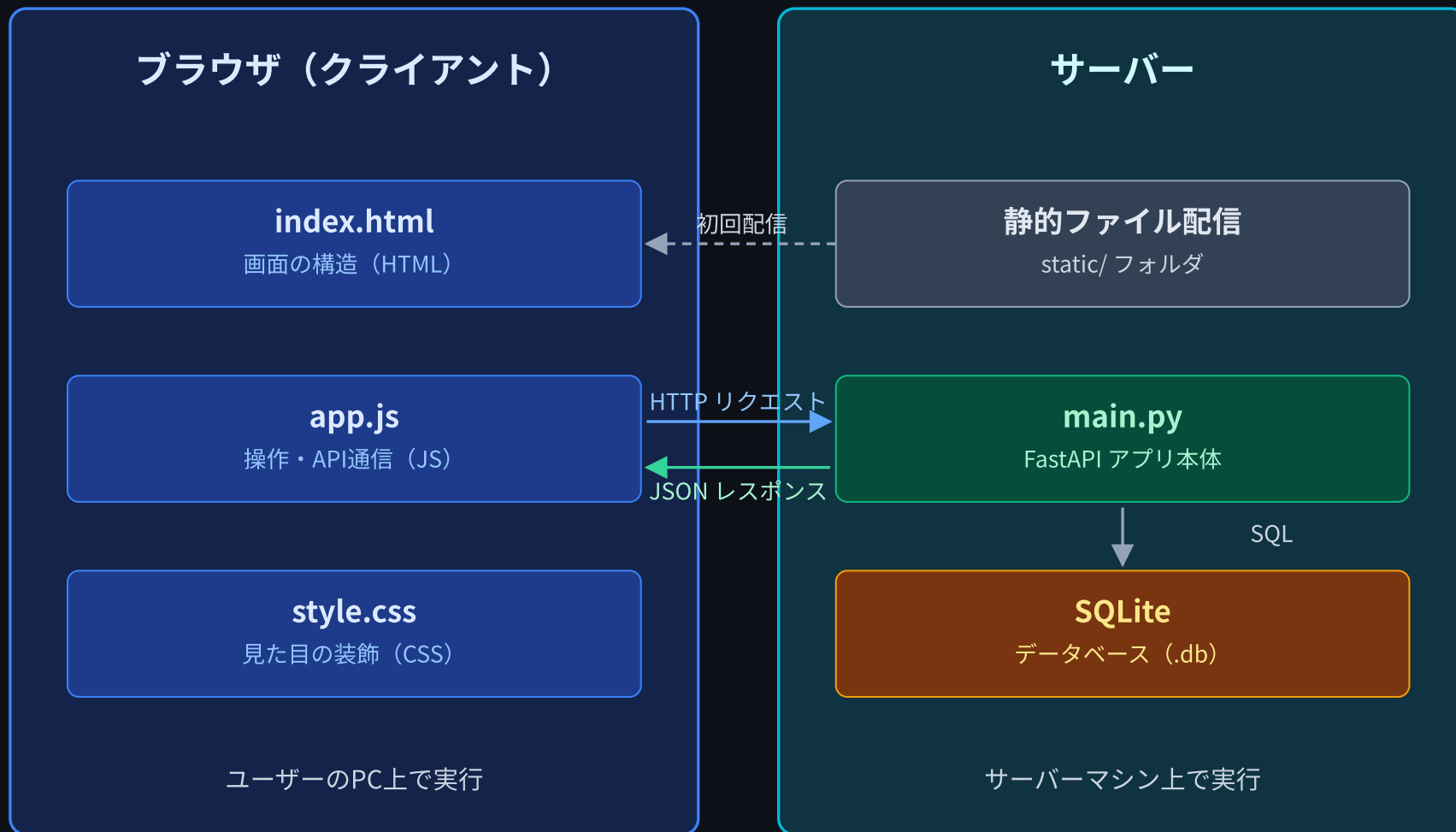
動作確認 → TODOアプリ完成

4. ブラウザで動作確認:
 - チェックボックスで完了切替
 - 削除ボタンでTODO削除
5. ページをリロードして、データが残っていることを確認！

Git commit & push

6. `git add . && git commit -m "第7回: TODOアプリフルスタック完成" && git push`

完成！TODOアプリの全体像



6. CORS（発展）

もしフロントエンドを別のサーバーに分けたら？

今回は 画面もAPIも同じサーバー（同一オリジン） から配信したので、

`fetch("/todos")` がそのまま動きました。

でも実務では、フロントエンドとバックエンドを 別々のサーバー（別オリジン） に分けることがあります。そのとき必要になるのが **CORS** の知識です。

オリジン = プロトコル + ホスト名 + ポート番号 の組み合わせ

`http://localhost:5500` ← フロントエンド（画面だけを配信するサーバー）

`http://localhost:8000` ← バックエンド（FastAPIサーバー）

↑ ↑
プロトコル ポートが違う！ → 異なるオリジン

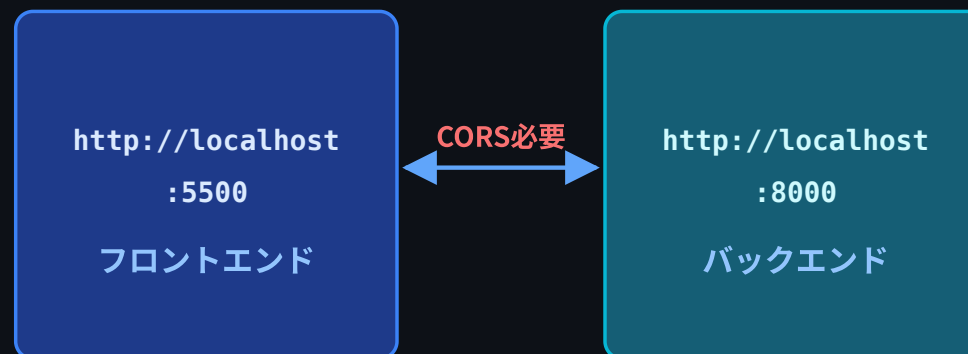
ブラウザはセキュリティのため、異なるオリジンへの通信をデフォルトでブロックする。
これが **CORS (Cross-Origin Resource Sharing)** の仕組み。

同一オリジン vs 別オリジン

今回（同一オリジン → CORS不要）

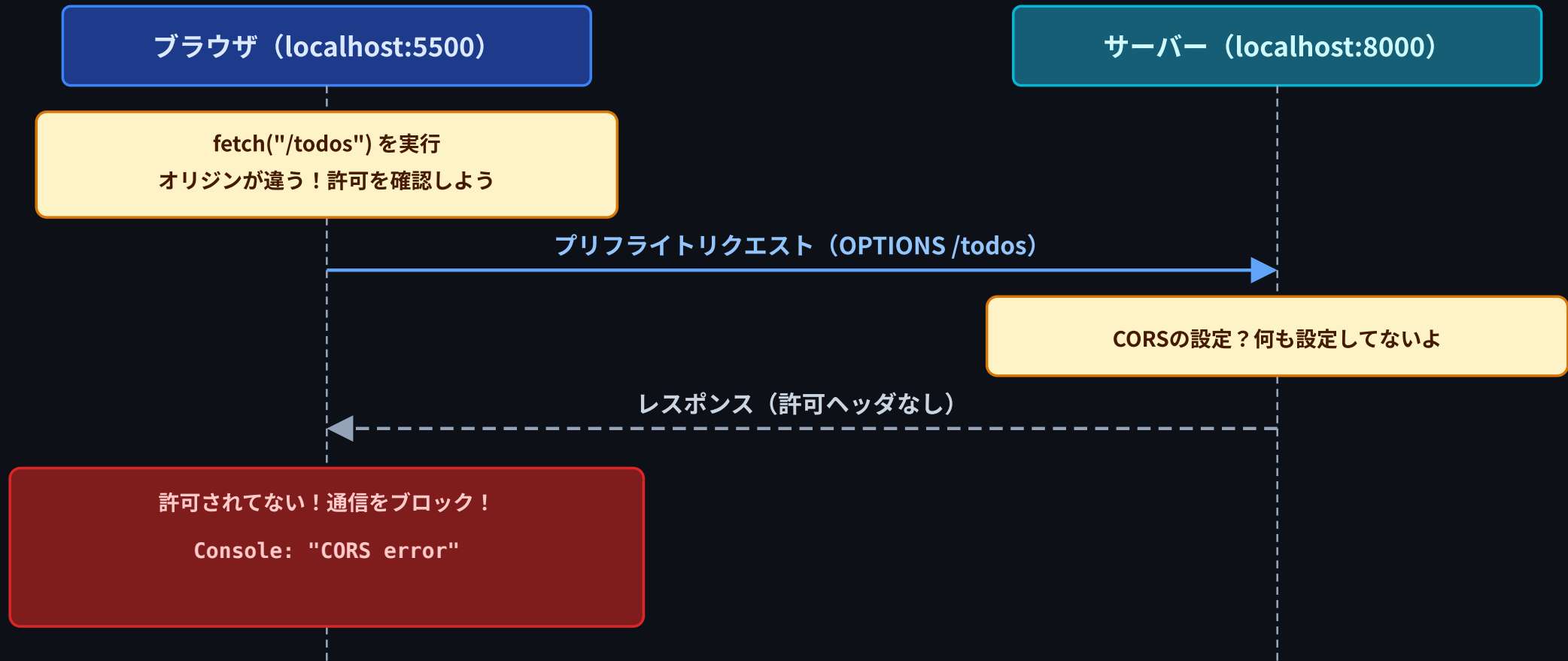


別サーバーに分けた場合（別オリジン → CORS必要）



静的配信で同一オリジンにまとめれば、そもそもCORSは要らない。
フロントとバックを分けるときだけ、CORSの設定が必要になる。

CORSエラーが起きる流れ



CORSの解決方法

FastAPIでCORSMiddlewareを設定する

```
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

# CORS設定を追加
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],           # どのオリジンからのアクセスを許可するか
    allow_credentials=True,       # "*" = すべて許可（開発時のみ推奨）
    allow_methods=["*"],         # どのHTTPメソッドを許可するか
    allow_headers=["*"],         # どのヘッダを許可するか
)
```

設定後の流れ:

サーバーが「このオリジンからのアクセスは許可します」とレスポンスヘッダに含める

→ ブラウザが通信を許可 → エラー解消

CORS解決後の通信フロー



実習6（発展）：CORS設定を体験する

やること

完成版の `todo-app` にも、この **CORS設定**が入っています。
将来フロントを別オリジンに分けても動くようにするための備えです。

1. `exercise/main.py` の CORS設定の TODO コメントを実装:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

2. サーバーを再起動しても、同一オリジンのアプリはこれまで通り動くことを確認
3. 追加したら再度 `git add . && git commit && git push`

発展課題: 別オリジン（別ポート）からAPIを呼ぶとどうなるか、CORSエラーの仕組みを調べてみよう

今回のまとめ

学んだこと	ポイント
全体アーキテクチャ	ユーザー操作 → JS → HTTP → FastAPI → DB → JSON → DOM更新
静的ファイル配信	<code>StaticFiles</code> でHTML/CSS/JSを配信。画面もAPIも同一オリジン
Fetch API (GET)	<code>fetch("/todos")</code> でデータ取得。 <code>async/await</code> で非同期処理
Fetch API (POST)	<code>method:"POST"</code> , <code>headers</code> , <code>body:JSON.stringify()</code>
PUT / DELETE	完了切替と削除もfetchで実装。操作後に一覧再取得
CORS（発展）	別オリジンに分けたときの通信制限。 <code>CORSMiddleware</code> で解決

次回予告: 第8回 セキュリティの基礎 & 総仕上げ

- XSS（クロスサイトスクリプティング）攻撃と対策
- SQLインジェクション攻撃と対策
- 入力バリデーション（Pydantic）
- エラーハンドリング
- TODOアプリの最終完成版

今回作ったTODOアプリに「安全対策」を施して完全体にします！

提出物

実習で穴埋め・実装を完了させたファイルをフォームから提出してください:

1. `flow.md` のGitHubのURL

- 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session07/exercise/flow.md`

2. `main.py` のGitHubのURL

- 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session07/exercise/main.py`

3. `app.js` のGitHubのURL

- 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session07/exercise/static/app.js`